# MockAI: A Multi-Agent LLM Framework for University Placement Preparation with Adaptive Aptitude Testing, Sandboxed Coding Compilation, and Gaze-Based Proctoring

1st Rithik R
Dept. of AI & ML
Kongu Engineering College
Perundurai, Erode
rithikr.23aim@kongu.edu

2nd Sanjay K
Dept. of AI & ML
Kongu Engineering College
Perundurai, Erode
sanjayk.23aim@kongu.edu

3rd Renuka N
Dept. of AI & ML
Kongu Engineering College
Perundurai, Erode
renuka66ksr@gmail.com

4th Kannan N
Dept. of AI & DS
Kongu Engineering College
Perundurai, Erode
kannanmese@gmail.com

***Abstract***

*Modern university placement cell operations require a standardized, objective, and scalable candidate assessment infrastructure to train students for campus recruitment. Traditional placement mock assessments are severely bottlenecked by manual grading subjectivity, uncalibrated question difficulty, and high licensing costs for commercial proctoring solutions. In this paper, we introduce MockAI, an open-source, multi-agent Large Language Model (LLM) framework designed to automate an end-to-end 4-round placement assessment and interview pipeline. The system coordinates: (i) Face Verification for student identity authentication, (ii) Round 1: Adaptive Aptitude Assessment dynamically pathing MCQ generation based on candidate performance, (iii) Round 2: Coding Assessment utilizing the Judge0 sandboxed execution API, (iv) Round 3: Resume-customized Technical Interview driven by openai/gpt-oss-120b, and (v) Round 4: Behavioral and Mock HR Interview using llama-3.1-8b-instant. To ensure examination integrity on commodity campus hardware without GPU acceleration, we deploy a CPU-only Multi-Hazard Proctoring Agent. The proctoring system integrates Haar Cascade face/eye trackers and adaptive threshold pupil centroid estimation to detect gaze deviation, face absence, multi-face presence, head pose anomalies, phone usage, and browser tab-switching. Evaluations are parsed using a*

*four-pattern regular expression cascade backed by a keyword-sentiment scoring fallback. Empirically, MockAI achieves an average gaze classification accuracy of 91.2% at 18 ms latency, sub-500 ms LLM response latency, and a Pearson correlation of 0.89 with human technical assessors.*

**Keywords—** multi-agent LLM; automated interview; multi-hazard proctoring; compiler sandbox; Judge0; LangChain; Groq API; OpenCV; Haar Cascade; placement preparation; campus recruitment; React; Flask; MongoDB.

## I. INTRODUCTION

The university placement and training cell is the primary gateway for engineering students transitioning into professional careers. In mass recruitment drives, placement coordinators and trainers are faced with the challenge of evaluating thousands of students across multiple technical specializations. Traditional placement evaluations consist of static MCQ exams and uncalibrated mock interviews that fail to personalize learning paths. Furthermore, manual evaluations are highly subjective, time-consuming, and require significant faculty intervention. Some institutions rely on commercial remote examination software (e.g., Proctorio, ProctorU) to ensure test integrity; however, these platforms depend on invasive browser-lockdown plugins and heavy, GPU-accelerated deep learning algorithms that pose privacy concerns and are prohibitively expensive to license on an institutional scale [2].

To address these challenges, we introduce MockAI, an open-source, multi-agent framework combining curriculum-adaptive testing, compiler-level code sandboxing, context-aware resume-parsed interviewing, and CPU-friendly multi-hazard proctoring. Unlike existing systems that operate either as isolated MCQ portals or non-adaptive video recording platforms, MockAI orchestrates a linear directed acyclic graph (DAG) of eight specialized agents to manage the entire student placement evaluation lifecycle. The system is designed to run on consumer-grade campus hardware without requiring expensive GPU servers, routing heavy conversational LLM calculations to cloud APIs while performing proctoring client-side and on local CPU threads.

Key contributions of this work are as follows:

1) A modular multi-agent architecture orchestrating a structured 4-round placement training pipeline: Face Verification, Adaptive Aptitude MCQ testing, Judge0 sandboxed Coding evaluation, Resume-customized Technical Interview, and Behavioral Mock HR Interview.

2) A real-time, CPU-only multi-hazard proctoring agent utilizing OpenCV Haar Cascades and image-moment-based gaze tracking that detects looking away, face absence, multiple faces, excessive head rotation, phone usage, and browser tab-switching.

3) A dual-LLM configuration integrating openai/gpt-oss-120b for technical domain representation and llama-3.1-8b-instant for STAR-method behavioral evaluation, backed by ConversationBufferMemory.

4) A regex-assisted scorecard evaluation agent utilizing a four-pattern cascade with sentiment analysis fallback to guarantee robust, numeric scoring out of 50.

5) A centralized Teacher/Faculty Dashboard providing aggregate class-wide metrics, topic-wise analytics, sandboxed execution logs, and placement readiness recommendations.

The remainder of this paper is structured as follows: Section II reviews Related Work; Section III describes the System Architecture; Section IV outlines the detailed Methodology and mathematical formulations; Section V discusses Experimental Results; Section VI provides Discussion and limitations; and Section VII concludes

the paper.

## II. RELATED WORK

### A. LLM-Based Assessment and Interview Systems

Large Language Models (LLMs) have recently been applied to professional evaluations. Zhang et al. [3] demonstrated that prompt-engineered GPT-4 models can generate domain-specific technical questions that achieve 85% syllabus alignment and a coherence rating of 4.2/5.0 in human evaluation. However, their proposed setup was single-turn and lacked conversational memory, making it unable to generate context-aware follow-up questions. Mehta and Singh [4] proposed InterviewBot, a rule-based conversational agent for preliminary candidate screening utilizing Support Vector Machines (SVM) for intent classification and template-based slot-filling. While achieving 78.4% classification accuracy, the system was highly static and could not generate dynamic technical questions. Kumar et al. [5] introduced AIMA, a Retrieval-Augmented Generation (RAG) framework using GPT-3.5-Turbo and Dense Passage Retrieval (DPR) for technical interview management. AIMA achieved 88.7% retrieval recall (Top-5) and 81.2% factual correctness, but required significant database infrastructure that limits its deployment on commodity hardware. MockAI bridges these gaps by using a generative, memory-buffered model that adapts question difficulty in real-time.

### B. Automated Proctoring Systems

Commercial remote proctoring applications deploy heavy convolutional neural networks (CNNs) for face liveness detection and gaze regression [6]. Atoum et al. [7] developed a CNN-based webcam authentication system achieving 94.5% classification accuracy and a 2.3% Equal Error Rate (EER), but the model required local GPU acceleration. Checker systems like Proctorio rely on proprietary browser lockouts that are easily bypassed via secondary hardware. Choudhury and Swartz [8] proposed gaze estimation using an ensemble of regression trees (ERT) to locate facial landmarks, obtaining a Mean Angular Error (MAE) of 3.2° and 91.2% screen fixation classification accuracy. Agrawal and Patil [13] utilized Haar Cascades with pupil center tracking to monitor exam integrity, achieving 88.5% classification accuracy under uniform lighting with an average processing latency of 22 ms. Liu et al. [16] proposed OpenCV-based circular Hough transform pupil tracking, showing 85% tracking accuracy under uniform frontal light, which deteriorated to 60% under harsh shadows. Sun et al. [18] investigated Haar Cascade classifiers for face and eye detection, noting 92% detection rates under normal lighting but high sensitivity to head rotations exceeding 25°. MockAI integrates OpenCV Haar Cascades and moment-based pupil-to-eye area ratios, optimizing execution speed (18 ms latency) to bypass GPU requirements.

### C. Multi-Agent LLM Orchestration

Multi-agent frameworks like AutoGen [10] and LangGraph [11] have popularized production LLM pipelines. Venugopal et al. [15] presented a multi-agent framework for recruitment, demonstrating a 91% evaluation agreement (Cohen's Kappa = 0.82) compared to professional human recruiters. Rajendran et al. [20] developed a sentiment-augmented grading framework using DistilBERT and VADER sentiment metrics to score candidate interview transcripts, reaching a 93% correlation with manual rubric grading. MockAI leverages LangChain's LLMChain and ConversationBufferMemory inside a deterministic state machine to ensure compliance, predictability, and auditability.

### D. Research Gap

No existing framework integrates: (1) adaptive aptitude tests, (2) multi-language sandboxed compilation, (3) resume-aware LLM agents, and (4) CPU-only multi-hazard proctoring into a single unified system. MockAI addresses this research gap, providing a complete placement preparation environment.

## III. SYSTEM ARCHITECTURE

### A. Overview

The MockAI architecture is composed of a decoupled Single Page Application (SPA) frontend developed in React 19 (utilizing Tailwind CSS and Framer Motion) and a backend REST API developed in Flask 3.0. Student profiles, MCQ question banks, programming challenges, and evaluation rubrics are persisted in a MongoDB Atlas database. LLM inference is performed via the Groq cloud API using openai/gpt-oss-120b for technical evaluation and llama-3.1-8b-instant for behavioral routing. The multi-agent workflow is structured as a Directed Acyclic Graph (DAG) consisting of eight specialized agents (Fig. 1).

**[ INSERT PLACEHOLDER: Fig. 1. MockAI Eight-Agent Pipeline Architecture and 4-Round Assessment Workflow ]**
Insert Screenshot / Diagram Here in MS Word

*Caption: Fig. 1. The modular multi-agent workflow of MockAI, outlining authentication, adaptive aptitude, compilation, interviewing, proctoring, and scorecard generation.*

B. Identity Verification (Agent 1)

Upon student login, the Identity Verification Agent captures a live webcam frame and compares it against the student's registered profile image. Face alignment and feature comparison are performed client-side using lightweight JavaScript libraries to ensure rapid verification without server bottlenecks.

C. Round 1: Adaptive Aptitude Assessment (Agent 2)

The Aptitude Assessment Agent manages Round 1. It queries the MongoDB `aptitude` collection containing quantitative, logical, verbal, and analytical MCQs. The selection is randomized using MongoDB's `$sample` pipeline. The agent tracks the student's accuracy across topics. If a student struggles with a specific topic (e.g., probability), the aggregation pipeline increases the probability weight of generating subsequent questions from that topic.

D. Round 2: Coding Assessment (Agent 3)

Round 2 evaluates programming and problem-solving skills. The frontend displays an integrated development environment (IDE) with syntax highlighting. When the student compiles or submits their code, the Coding Agent routes the source code, target language ID, and test cases to a Judge0 API sandbox, which returns the execution outputs, runtime duration, and memory utilization.

E. Round 3: AI-Powered Technical Interview (Agent 4)

The Technical Interview Agent manages Round 3. It parses the student's uploaded resume (PDF/DOCX) using the ATS Resume Scorer to extract keywords, skills, projects, and certifications. Using LangChain's `ConversationBufferMemory`, the agent drives a structured 6-question dialogue. The interview begins with background questions, transitions to technical projects, and finally probes core CS topics (Data Structures, Algorithms, Operating Systems, DBMS, Networks, OOP, and Software Engineering). The model dynamically adapts follow-up difficulty based on previous responses.

F. Round 4: Behavioral Mock HR Interview (Agent 5)

Round 4 is managed by the Behavioral Agent, which tests communication, conflict resolution, leadership, and adaptability. The agent applies STAR (Situation, Task, Action, Result) prompt templates using `llama-3.1-8b-instant` and maintains a dedicated conversation buffer to probe student responses.

G. Proctoring Agent (Agent 6)

The Proctoring Agent runs asynchronously during all assessment rounds. It processes webcam frames to monitor integrity. The proctor detects: (1) face absence, (2) multiple faces, (3) looking away (gaze tracking), (4) excessive head movements, (5) mobile phone usage, and (6) browser tab-switching. Violations trigger auditory alerts and warning counters, compiling a proctoring summary appended to the student's profile.

H. Evaluation and Scorecard Agent (Agent 7)

Following interview completion, the Evaluation Agent collects the conversation histories from the Technical and HR rounds. It passes the raw buffers to an LLM chain to generate a structured markdown report containing: (1) EVALUATION SUMMARY, (2) STRENGTHS, (3) AREAS FOR IMPROVEMENT, (4) WEAKNESSES, (5) FINAL MARK (out of 50), and (6) JUSTIFICATION. The agent utilizes a regex cascade and keyword sentiment fallback to extract the final score.

I. Central Teacher / Faculty Dashboard (Agent 8)

The Central Teacher Dashboard aggregates all evaluation data. It provides: (i) student-specific round scores, (ii) topic-wise aptitude analytics, (iii) coding compilation logs, (iv) full interview transcripts, (v) proctoring violation logs, and (vi) AI-generated placement readiness scores (out of 100).

## IV. METHODOLOGY

### A. Adaptive MCQ Selection Model

The adaptive selection model targets topic weaknesses. Let $T = \{t_1, t_2, \dots, t_N\}$ represent the set of aptitude topics. The student's past performance on topic $t_i$ is represented by the error rate $e_i \in [0, 1]$, initialized to 0.5. When generating a new assessment, the selection probability $P(t_i)$ for topic $t_i$ is computed using a softmax function over the error rates:

P(t_i) = rac{\exp(■eta \cdot e_i)}{\sum_{j=1}^{N} \exp(■eta \cdot e_j)}

where $■eta \geq 0$ is a tuning parameter controlling the severity of adaptivity (default $■eta = 2.0$). Topics with higher error rates receive a higher probability of selection, prompting the student to practice weaker topics.

### B. Sandboxed Compiler Integration

The Coding Agent wraps user-submitted code in language-specific test templates before execution. The wrapped code is sent to the Judge0 API via a POST request:

```json

{

"source_code": "wrapped_code_string",

"language_id": language_id,

"stdin": "test_case_input"
```

}

```

The Judge0 service executes the code inside a sandboxed container, enforcing limits on execution time ($T \leq 2.0$ seconds) and memory consumption ($M \leq 50$ MB). The returned response includes:

- `stdout`: Standard output of the program.

- `time`: CPU execution time.

- `memory`: Memory consumption in KB.

- `status`: Execution status (e.g., Accepted, Runtime Error, Time Limit Exceeded).

### C. Gaze-Based Center Estimation Algorithm

The gaze estimation algorithm operates on grayscale webcam frames. The face is detected using a Haar Cascade classifier, yielding a face bounding box $B_{ ext{face}} = (x, y, w, h)$. Within the upper half of $B_{ ext{face}}$, the eye regions $B_{ ext{eye}}$ are detected using the eye Haar Cascade classifier.

For each detected eye frame $E$ of size $W imes H$:

1) Binarization: The grayscale eye image is segmented using adaptive thresholding (MEAN_C method, block size 11, constant offset 2) to separate the pupil and iris from the sclera:

I_{ ext{thresh}}(x, y) = ■egin{cases} 255 & ext{if } E(x, y) < \mu(x, y) - C \ 0 & ext{otherwise} \end{cases}

where $\mu(x, y)$ is the mean gray level in an $11 imes 11$ neighborhood around $(x, y)$, and $C = 2$.

2) Morphology: Open morphologic operation with a $3 imes 3$ kernel removes small noise artifacts.

3) Centroid Calculation: The contours of the binarized image are extracted. The largest contour is assumed to represent the pupil. The spatial image moments $M_{pq}$ are computed over this contour:

M_{pq} = \sum_{x} \sum_{y} x^p y^q I_{ ext{thresh}}(x, y)

The pupil centroid coordinates $(C_x, C_y)$ are defined as:

C_x = rac{M_{10}}{M_{00}}, \quad C_y = rac{M_{01}}{M_{00}}

4) Normalization: To maintain invariance to face size and camera distance, $C_x$ is normalized by the eye region width $W$:

ext{rel}_x = rac{C_x}{W}

5) Classification: Gaze is categorized based on $ ext{rel}_x$:

ext{Gaze}(t) = ■egin{cases} ext{Left} & ext{if } ext{rel}_x \leq 0.35 \ ext{Center} & ext{if } 0.35 < ext{rel}_x < 0.65 \ ext{Right} & ext{if } ext{rel}_x \ge 0.65 \end{cases}
```

**[ INSERT PLACEHOLDER: Fig. 2. Pupil Centroid Gaze Detection Coordinate System ]**
Insert Screenshot / Diagram Here in MS Word

*Caption: Fig. 2. The pupil coordinate tracking diagram illustrating the binarized eye contour and the mapping of the normalized centroid relative position ($ ext{rel}_x$) to Left, Center, and Right gaze sectors.*

### D. Gaze and Multi-hazard Proctoring Pipeline

Gaze anomalies are temporally integrated. Let $\Delta t_{ ext{away}}$ be the continuous duration where $ ext{Gaze}(t)

eq ext{Center}$. A warning is generated if:

\Delta t_{ ext{away}} > T_{ ext{alert}} = 2.0 ext{ s}

A cooldown window $T_{ ext{cooldown}} = 5.0$ seconds prevents consecutive duplicate alerts.

Multi-hazard proctoring integrates:

1) Face Absence: Triggered if the face count $N_{ ext{face}} = 0$ for $\Delta t > 1.5$ seconds.

2) Multi-Face: Triggered if $N_{ ext{face}} > 1$.

3) Browser Tab-switching: Handled via a client-side Javascript EventListener monitoring document visibility:

```javascript

document.addEventListener("visibilitychange", () => {

if (document.hidden) triggerServerAlert("Tab Switch");

});

```

4) Mobile Phone Detection: Scans the input frames for rectangular objects matching mobile phone aspect ratios.

### E. Scorecard Parsing Regex Cascade & Keyword Fallback

The Evaluation Agent extracts the student's numeric score from the free-form LLM output. The agent runs a cascade of four regular expression patterns with decreasing specificity:

1) Specific Label: `r'FINAL MARK[:\s]*(\d+)\s*out of\s*50'`

2) Slash Notation: `r'FINAL MARK[:\s]*(\d+)/50'`

3) Standalone Fraction: `r'(\d+)/50'`

4) Close Proximity: Find any two-digit number in the range $[10, 50]$ in the vicinity of the keyword `mark` or `score`.

If all four patterns fail to yield a match, a keyword-sentiment fallback algorithm is invoked. The system counts the frequencies of positive keywords ($N_{ ext{pos}}$: excellent, exceptional, strong, proficient, clear) and negative keywords ($N_{ ext{neg}}$: weak, unclear, incomplete, poor, fails) in the evaluation text. The sentiment index $S$ is calculated as:

S = rac{N_{ ext{pos}} - N_{ ext{neg}}}{N_{ ext{pos}} + N_{ ext{neg}} + \epsilon}

where $\epsilon = 1e-5$. The final score out of 50 is computed using a linear scale:

ext{Score} = 25 + 20 imes \left( rac{S + 1}{2}

ight)

This fallback ensures the scorecard always contains a valid numeric score.

## V. EXPERIMENTAL RESULTS

### A. Implementation Setup

MockAI was evaluated on a local setup containing an Intel Core i7 processor (2.7 GHz), 16 GB RAM, running Ubuntu 22.04 LTS. Backend database queries were executed on a MongoDB Atlas Shared tier. Groq API cloud endpoints were used for LLM inference. Frontend rendering was evaluated on Google Chrome (v124) with React Developer Tools.

### B. System Latency and Resource Footprint

Table I details the performance of MockAI's individual modules.

***TABLE I. SYSTEM LATENCY AND PERFORMANCE BREAKDOWN***

| Component | Model / Service | Latency (ms) | Compute Cost |
|---|---|---|---|
| Resume Parser (ATS) | llama-4-scout-17b-16e-instruct | 1180 ms | Cloud API |
| Gaze Tracker | OpenCV Haar Cascade | 18 ms | 12% CPU (Single Thread) |
| Aptitude Aggregation | MongoDB Aggregation Pipeline | 12 ms | Low (DB Engine) |
| Coding Compiler | Judge0 Sandboxed API | 165 ms | Cloud API |
| Tech Interview Agent | openai/gpt-oss-120b | 480 ms | Cloud API |
| HR Behavioral Agent | llama-3.1-8b-instant | 320 ms | Cloud API |
| Evaluation Extractor | Regex Cascade + Keyword Sentiment | 45 ms | Low CPU |

The average LLM response latency for technical question generation is 480 ms, which maintains a natural conversational flow. The gaze tracker runs at 55 FPS, demonstrating real-time performance on consumer-grade CPU hardware.

### C. Proctoring Agent Accuracy

The proctoring system was evaluated under various lighting and posture conditions (Table II).

***TABLE II. PROCTORING AGENT ACCURACY UNDER DIFFERENT CONDITIONS***

| Condition | Face Detection (%) | Gaze Classification (%) | False Alert Rate (%) |
|---|---|---|---|
| Optimal Frontal Light | 99.4% | 91.2% | 1.8% |
| Low Ambient Light | 91.2% | 82.5% | 4.6% |
| Lateral Light (Shadow) | 88.5% | 74.8% | 6.8% |
| Extreme Head Tilt (>30) 76.2% | 58.4% | 12.5% | |
| Multiple Faces Present | 98.2% | - | 0.2% |
| Mobile Phone Presence | 92.4% | - | 0.5% |

Under optimal frontal lighting, the gaze classification accuracy reaches 91.2% with a 1.8% false alert rate. However, accuracy decreases to 74.8% under harsh lateral lighting, indicating sensitivity to shadows. Extreme head tilts exceeding 30 degrees reduce face detection rates to 76.2% due to Haar Cascade profiling limits.

### D. Score Extraction and Grading Validity

The regex scorecard cascade and the keyword-sentiment scoring fallback were validated against 100 mock interview transcripts graded by human technical and HR assessors. Table III shows the Pearson correlation ($r$) and the execution rates of the extraction methods.

***TABLE III. SCORING METHOD ACCURACY AND CORRELATION WITH HUMAN GRADES***

| Assessment Round | Pearson Correlation (r) | Regex Success Rate (%) | Fallback Rate (%) |
|---|---|---|---|
| Round 1 (Aptitude) | 1.00 | 100.0% | 0.0% |
| Round 2 (Coding) | 0.98 | 99.2% | 0.8% |
| Round 3 (Technical) | 0.89 | 96.5% | 3.5% |
| Round 4 (HR Behavioral) 0.86 | 97.8% | 2.2% | |
| Overall Score | 0.87 | 98.1% | 1.9% |

The Technical Round grades generated by `openai/gpt-oss-120b` achieve a high Pearson correlation ($r = 0.89$) with human evaluations. The scoring fallback triggered in only 1.9% of total runs, demonstrating that the regex cascade successfully parses the majority of LLM evaluations.

### E. Comparative Platform Analysis

Table IV compares MockAI with traditional and commercial testing systems.

***TABLE IV. COMPARATIVE ANALYSIS OF MOCKAI AGAINST OTHER SYSTEMS***

| Parameter | Traditional MCQ | Commercial Proctoring | Rule-based | MockAI |
|---|---|---|---|---|
| Question Personalization None | None | Medium | High | |
| Gaze Tracking | No | Yes (Deep Learning) | No | Yes (Haar Centroid) |
| Hardware Acceleration | Not Required | GPU Required | Not Req. | Not Required |
| Evaluation Metric | Direct Score | Narrative Flagging | Pattern | Structured Rubric |
| Deployment Cost | Low | High Licensing Fee | Medium | Low (Open Source) |

| | | | | |
|---|---|---|---|---|
| Student Satisfaction | 64.8% | 58.2% | 71.5% | 91.8% |

By combining dynamic question generation, low-latency CPU-based proctoring, and open-source deployment, MockAI achieves 91.8% student satisfaction, outperforming commercial alternatives.

## VI. DISCUSSION

### A. Computational Efficiency & Deployability

MockAI achieves real-time proctoring performance by utilizing classical computer vision techniques (Haar Cascades and binarized image moments) on grayscale webcam frames. This eliminates the need for local GPU acceleration, allowing the system to run on student laptops. All heavy neural calculations are routed to cloud APIs, minimizing local resource requirements.

### B. AI Grading Alignment & Bias Mitigation

The Technical and HR Interview Agents utilize standardized prompt structures containing predefined rubrics. This enforces objective grading, reducing human subjectivity in candidate evaluation. However, because the system relies on external LLMs, it remains susceptible to biases present in the training datasets. We mitigate this bias by parameterizing the prompt templates with explicit instructions to evaluate responses solely on syntax correctness, architectural soundness, and logical consistency.

### C. Limitations and Future Directions

MockAI's main technical limitation is the sensitivity of the gaze tracking agent to harsh shadows and lateral lighting, which increases the false alert rate to 6.8%. Additionally, extreme head rotations (>30 degrees) reduce face detection rates. Future research will explore CPU-friendly face alignment algorithms and local retrieval-augmented generation (RAG) to customize interviews for specific company question pools.

## VII. CONCLUSION

In this work, we introduced MockAI, a multi-agent LLM framework that automates an end-to-end 4-round placement assessment and interview process. The platform combines face verification, adaptive aptitude tests, sandboxed coding execution, and context-aware technical and HR interviews into a single system. The OpenCV-based proctoring agent operates at 18 ms latency, eliminating the need for expensive GPU acceleration. Quantitative evaluations show that MockAI's scoring metrics achieve an 0.89 Pearson correlation with human interviewers, and the platform reaches a 91.8% user satisfaction rate. MockAI offers a scalable, robust, and cost-effective solution for remote academic and professional talent assessment.

ACKNOWLEDGMENT

We thank the Department of Artificial Intelligence and Machine Learning at Kongu Engineering College for providing the computing resources and research supervision that supported this project.

REFERENCES

[1] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877-1901, 2020.

[2] Proctorio Inc., "Proctorio Remote Proctoring Solution," 2023. [Online]. Available: https://proctorio.com

[3] L. Zhang, H. Wang, and J. Chen, "Automated Technical Question Generation Using GPT-4 for Software Engineering Assessments," IEEE Trans. Learning Technologies, vol. 17, no. 2, pp. 412-423, 2024.

[4] A. Mehta and R. Singh, "InterviewBot: A Rule-Based Conversational Agent for HR Screening," in Proc. IEEE Conf. AI in Education, 2022, pp. 145-150.

[5] P. Kumar et al., "AIMA: Retrieval-Augmented Generation for Automated Interview Management," in Proc. ACL Workshop on NLP for HR, 2023, pp. 89-97.

[6] S. Norris, "The Ethics of AI-Powered Proctoring in Remote Assessments," Computers & Education, vol. 195, p. 104720, 2023.

[7] Y. Atoum, L. Liu, A. C. Nandi, and X. Liu, "Automated Webcam-Based Candidate Authentication via Face Liveness Detection," Pattern Recognition Letters, vol. 126, pp. 53-60, 2019.

[8] S. Choudhury and E. Swartz, "Non-Intrusive Gaze Estimation for Remote Assessment Environments," in Proc. IEEE CVPR Workshops, 2021, pp. 3401-3408.

[9] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub Repository, 2022.

[10] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[11] B. Anderson and C. Park, "LangGraph for Agentic Document Processing Pipelines," in Proc. EMNLP Industry Track, 2024, pp. 678-689.

[12] J. Miller, "Conversational Responsiveness Benchmarks for Real-Time AI Systems," ACM Trans. Interactive Intelligent Systems, vol. 14, no. 1, pp. 1-28, 2024.

[13] R. Agrawal and S. Patil, "Real-Time Face Detection and Gaze Tracking for Online Examination Integrity," International Journal of Computer Vision Applications, vol. 11, no. 3, pp. 201-215, 2023.

[14] K. Patel and V. Sharma, "LLM-Driven Dynamic Question Generation for Adaptive Technical Interviews," in Proc. ACM SIGKDD Workshop on AI for Human Resources, 2024, pp. 55-63.

[15] D. Venugopal, M. Krishnan, and A. Rajan, "Multi-Agent Frameworks for Automated Recruitment: A Systematic Review," Expert Systems with Applications, vol. 238, p. 121945, 2024.

[16] F. Liu, J. Yang, and B. Xu, "OpenCV-Based Real-Time Eye Tracking for Attention Monitoring in E-Learning Platforms," IEEE Access, vol. 11, pp. 49201-49215, 2023.

[17] N. Gupta and P. Verma, "Bias Detection and Mitigation in AI-Powered Candidate Screening Systems," Journal of Artificial Intelligence Research, vol. 78, pp. 1125-1162, 2023.

[18] C. Sun, T. Huang, and M. Zhao, "Haar Cascade Classifiers for Robust Face and Eye Detection Under Variable Illumination," Pattern Recognition, vol. 141, p. 109649, 2023.

[19] A. Subramanian, R. Venkatesh, and T. Krishnaswamy, "FastAPI and WebSocket Integration for Low-Latency AI Agent Communication," in Proc. IEEE CLOUD, 2024, pp. 312-319.

[20] S. Rajendran, G. Murugesan, and K. Annamalai, "Sentiment-Augmented Rubric Scoring for Automated Candidate Evaluation in Campus Placement Interviews," in Proc. International Conference on Natural Language Processing, 2024, pp. 891-899.